

Wednesday 5:46 PM



lab_one.pdf
PDF

Here is my lab, run me though what we doing, what language, what is the task, how can i go about it



How do the files look from a glance, also why is there a
claude.md file?

Ari, the next feature is **overdue task highlighting**. The brief requires overdue tasks to be visibly indicated, but `overdue` must not become a selectable status or database column.

`lab_one`

1. Update the Prisma import

In `src/app/page.tsx`, change:

TypeScript

to:

```
import {
  Prisma,
  TaskStatus,
} from "@generated/prisma/client";
```

2. Add the overdue function

Place this above the `Home` component:

```
function isTaskOverdue(
  dueDate: Date,
  status: TaskStatus,
) {
  return (
    dueDate.getTime() < Date.now() &&
    status !== TaskStatus.COMPLETE
  );
}
```

The rule is therefore:

```
Due date has passed
AND
Status is not Complete
```

A completed task should not be flagged overdue, even if its due date is in the past.

3. Update each active task item

Find this section:

```
{activeTasks.map((task) => (
  <div key={task.id}>
    <div>
      <div>
```

Replace it with:

```
{activeTasks.map((task) => {
  const overdue = isTaskOverdue(
    task.dueDate,
    task.status,
  );
```

```

return (
  <li
    key={task.id}
    className={
      overdue
      ? "task-card task-card-overdue"
      : "task-card"
    }
  >

```

Inside the task, directly underneath the title:

TypeScript

add:

```

{overdue && (
  <p className="overdue-badge">
    Overdue
  </p>
)}

```

At the end of that mapped task, you currently have something like:

```

</li>
'''
Task Card

```

Because we added braces to the mapping function, change it to:

```

</li>
);
}}

```

The full basic structure should resemble:

```

<ul>
  {activeTasks.map((task) => {
    const overdue = isTaskOverdue(

```

```

    task.dueDate,
    task.status,
  );

```

```

return (
  <li
    key={task.id}
    className={

```

TypeScript

```

      : "task-card"
    }
  >
  <h3>{task.title}</h3>

  {overdue && (
    <p className="overdue-badge">
      Overdue
    </p>
  )}

  <p>{task.description}</p>
  <p>Topic: {task.topic}</p>
  <p>Status: {formatStatus(task.status)}</p>
  <p>
    Due: {task.dueDate.toLocaleString()}
  </p>

  {/* Keep your edit and archive forms here */}
</li>
);
}}
</ul>

```

Do not remove your existing edit and archive forms—keep them inside the `<li>`.

4. Add basic styling

Open:

```
src/app/globals.css
```

Add this at the bottom:

CSS

```

.task-card {
  margin-bottom: 1rem;
  padding: 1rem;
  border: 1px solid #d1d5db;
  border-radius: 0.5rem;

```

```
}

.task-card-overdue {
  border: 2px solid #b91c1c;
  background-color: #fef2f2;
}

.overdue-badge {
  display: inline-block;
  margin: 0 0 0.75rem;
  padding: 0.25rem 0.5rem;
  border-radius: 0.25rem;
  background-color: #b91c1c;
  color: white;
  font-weight: 700;
}
```

5. Test the rule

Run:

```
PowerShell

npm run dev
```

Test these three cases:

Due date	Status	Expected
Yesterday	Todo	Overdue
Yesterday	In-Progress	Overdue
Yesterday	Complete	Not overdue
Tomorrow	Todo	Not overdue

Also confirm that the status dropdown still contains only:

```
Todo
In-Progress
Complete
```

There should be:

- No overdue option.
- No overdue database field.
- No Prisma migration for this feature.

Run the checks:

PowerShell

```
npm run lint  
npm run build
```

6. Commit the feature

Once everything works:

PowerShell

```
git add .  
git commit -m "Derive and display overdue task warnings"  
git push
```

This should be another clean, coherent commit. After this, the next major section is **automated testing with a disposable SQLite database**.

    Sources

Ari, that output is clean:

- ESLint passed.
- TypeScript passed.
- Production build compiled successfully.

- The `/` route is dynamic (`f`) because it reads task data from SQLite—expected, not an error.
- Environments: `.env` only means Next.js loaded your local `.env` ; it does **not** mean Git will commit it.

Commit the current feature set now:

PowerShell

```
git status
git add .
git commit -m "Add overdue indicators and refine task layout"
git push
```

Then begin testing:

PowerShell

```
npm install --save-dev vitest
```

Add this script to `package.json` :

JSON

```
"test": "vitest run"
```

Your scripts section should resemble:

JSON

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint",
  "test": "vitest run"
}
```

The lab requires at least three deterministic tests, with at least one covering archiving or the overdue rule, all runnable through one documented command. lab_one

  ...  Sources

Yes, Ari — **you're cool.**

You have:

- **5 passing tests**, exceeding the minimum of three.
- Overdue-rule coverage.
- A database-behaviour test.
- One command, `npm test`, running everything successfully.

The only thing to confirm is that `task-database.test.ts` uses a **temporary/disposable SQLite database**, not your normal `dev.db`, and cleans it up afterwards. That is specifically required for full testing marks. lab_one

Run the final checks:

PowerShell

```
npm run lint
npm run build
npm test
```

Then commit:

PowerShell

```
git add .
git commit -m "Add deterministic task rules and database tests"
git push
```

Next comes the README documentation and clean-clone verification.

  ...  Sources

RUN v4.1.10 C:/Users/USER-PC/Documents/coms3011a-lab1

✓ tests/task-rules.test.ts (4 tests) 3ms
✓ tests/task-database.test.ts (1 test) 132ms

Test Files 2 passed (2)

Tests 5 passed (5)

Start at 16:08:08

Duration 435ms (transform 88ms, setup 0ms, import
215ms, tests 136ms, environment 0ms)

Show more ▾

Yes, Ari — **everything is green:**

- 5/5 tests passed
- ESLint passed
- TypeScript passed
- Production build succeeded
- Dynamic / route is expected because it reads from SQLite

Commit this testing milestone:

PowerShell

```
git status
```

Confirm `.env` , `dev.db` , and any temporary test database are **not staged**, then:

PowerShell

```
git add .  
git commit -m "Add deterministic task rules and database tests"  
git push
```

The next phase is the **README documentation and clean-clone test**—the most dangerous part to neglect because the marker must be able to start the project using the README alone.



how do i reopen this proect

Ari, to reopen the project:

From PowerShell

PowerShell

```
cd "$HOME\Documents\coms3011a-lab1"  
code .
```

This opens the folder in VS Code.

Then open **Terminal** → **New Terminal** and run:

PowerShell

```
npm run dev
```

Open:

```
http://localhost:3000
```

From VS Code directly

1. Open VS Code.
2. Select **File** → **Open Folder**.
3. Choose:

```
C:\Users\USER-PC\Documents\coms3011a-lab1
```

4. Open a terminal and run:

PowerShell

```
npm run dev
```

Your tasks should still be present because they are stored in the local SQLite database. To stop the application, press **Ctrl + C** in the terminal.



ChatGPT can make mistakes. Check important info.